



Full Name: Jianping Ye

Email: yehdan@msn.com

Test Name: Mock Test

Taken On: 11 Aug 2025 18:31:22 IST

Time Taken: 14 min 5 sec/ 28 min

Invited by: Ankush

Invited on: 11 Aug 2025 18:31:15 IST

Skills Score:

Tags Score:

- Algorithms 100/100
- Core CS 100/100
- Graph Theory 100/100
- Medium 100/100
- problem-solving 100/100

100%

100/100

scored in **Mock Test** in 14 min 5 sec on 11 Aug 2025 18:31:22 IST

Recruiter/Team Comments:

No Comments.

	Question Description	Time Taken	Score	Status
Q1	Breadth First Search: Shortest Reach > Coding	12 min 52 sec	100/ 100	✓

QUESTION 1

Correct Answer

Score 100

Breadth First Search: Shortest Reach > Coding

Graph Theory Algorithms Medium

problem-solving Core CS

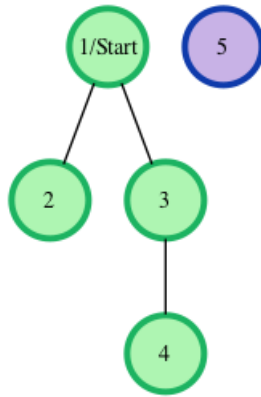
QUESTION DESCRIPTION

Consider an undirected graph where each edge weighs 6 units. Each of the nodes is labeled consecutively from 1 to n.

You will be given a number of queries. For each query, you will be given a list of edges describing an undirected graph. After you create a representation of the graph, you must determine and report the shortest distance to each of the other nodes from a given starting position using the *breadth-first search* algorithm (BFS). Return an array of distances from the start node in node number order. If a node is unreachable, return **−1** for that node.

Example

The following graph is based on the listed inputs:



$n = 5$ // number of nodes
 $m = 3$ // number of edges
 $edges = [1, 2], [1, 3], [3, 4]$
 $s = 1$ // starting node

All distances are from the start node **1**. Outputs are calculated for distances to nodes **2** through **5**: **[6, 6, 12, -1]**. Each edge is **6** units, and the unreachable node **5** has the required return distance of **-1**.

Function Description

Complete the `bfs` function in the editor below. If a node is unreachable, its distance is **-1**.

`bfs` has the following parameter(s):

- `int n`: the number of nodes
- `int m`: the number of edges
- `int edges[m][2]`: start and end nodes for edges
- `int s`: the node to start traversals from

Returns

`int[n-1]`: the distances to nodes in increasing node number order, not including the start node (-1 if a node is not reachable)

Input Format

The first line contains an integer **q**, the number of queries. Each of the following **q** sets of lines has the following format:

- The first line contains two space-separated integers **n** and **m**, the number of nodes and edges in the graph.
- Each line **i** of the **m** subsequent lines contains two space-separated integers, **u** and **v**, that describe an edge between nodes **u** and **v**.
- The last line contains a single integer, **s**, the node number to start from.

Constraints

- $1 \leq q \leq 10$
- $2 \leq n \leq 1000$
- $1 \leq m \leq \frac{n \cdot (n-1)}{2}$
- $1 \leq u, v, s \leq n$

Sample Input

```

2
4 2
1 2
1 3
1
3 1
2 3
2

```

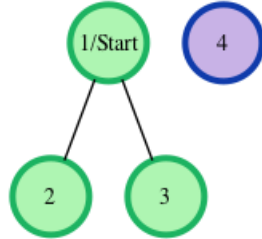
Sample Output

```
6 6 -1
-1 6
```

Explanation

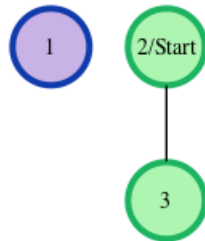
We perform the following two queries:

1. The given graph can be represented as:



where our *start* node, *s*, is node **1**. The shortest distances from *s* to the other nodes are one edge to node **2**, one edge to node **3**, and an infinite distance to node **4** (which it is not connected to). We then return an array of distances from node **1** to nodes **2**, **3**, and **4** (respectively): **[6, 6, -1]**.

2. The given graph can be represented as:



where our *start* node, *s*, is node **2**. There is only one edge here, so node **1** is unreachable from node **2** and node **3** has one edge connecting it to node **2**. We then return an array of distances from node **2** to nodes **1**, and **3** (respectively): **[-1, 6]**.

Note: Recall that the actual length of each edge is **6**, and we return **-1** as the distance to any node that is unreachable from *s*.

CANDIDATE ANSWER

Language used: **C++14**

```
1
2
3 // Create adjacent list from edge pairs
4 std::vector<std::set<int>> edges_to_adj(int n, vector<vector<int>> edges)
5 {
6     std::vector<std::set<int>> adj(n+1);
7     for (auto e : edges) {
8         adj[e[0]].insert(e[1]);
9         adj[e[1]].insert(e[0]); // Graph is undirected
10    }
11    return adj;
12 }
13
14 vector<int> bfs_dist(int n, const std::vector<std::set<int>>& adj, int
15 start_node)
16 {
17     std::vector<int> distances(n + 1, -1);
18     std::queue<int> q;
19
20     distances[start_node] = 0;
21     q.push(start_node);
22
```

```

23     while (!q.empty()) {
24         int u = q.front();
25         q.pop();
26
27         for (int v : adj[u]) {
28             if (distances[v] == -1) { // If neighbor 'v' is unvisited
29                 distances[v] = distances[u] + 6; // Assuming edge weight of 6
30                 q.push(v);
31             }
32         }
33     }
34
35     vector<int> out;
36     for (int i=1;i<=n;i++) {
37         if (i != start_node) out.push_back(distances[i]);
38     }
39
40     return out;
41 }
42
43 /*
44  * Complete the 'bfs' function below.
45  *
46  * The function is expected to return an INTEGER_ARRAY.
47  * The function accepts following parameters:
48  * 1. INTEGER n
49  * 2. INTEGER m
50  * 3. 2D_INTEGER_ARRAY edges
51  * 4. INTEGER s
52  */
53
54 vector<int> bfs(int n, int m, vector<vector<int>> edges, int s) {
55
56     auto adj = edges_to_adj(n, edges);
57
58     auto out = bfs_dist(n, adj, s);
59
60     return out;
61 }
62 }

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 1	Easy	Sample case	 Success	0	0.0085 sec	8.63 KB
Testcase 2	Medium	Hidden case	 Success	5	0.0119 sec	9 KB
Testcase 3	Medium	Hidden case	 Success	5	0.0399 sec	12.8 KB
Testcase 4	Hard	Hidden case	 Success	15	0.0087 sec	8.75 KB
Testcase 5	Hard	Hidden case	 Success	15	0.0104 sec	9 KB
Testcase 6	Hard	Hidden case	 Success	30	0.1938 sec	23.5 KB
Testcase 7	Hard	Hidden case	 Success	30	0.0269 sec	9.92 KB
Testcase 8	Easy	Sample case	 Success	0	0.0113 sec	8.75 KB

No Comments